

Operators

Operators :-

The operators are special symbol that perform specific operations on one, two or three operands and return a result.

Types of operators :-

1. Arithmetic Operator.
2. Comparison Operator / Relational Operator
3. Assignment Operator
4. Logical Operator
5. Bitwise Operator

Arithmetic Operator :-

Arithmetic Operators are used to perform arithmetic operations on the operands.

$\Rightarrow +, -, *, /, \%, **, //$

Assignment Operator :-

used to assign a value to variables.

$\Rightarrow =, +=, -=, *=, /=, \%=, **=, //=, \&=$

Ex:-

$x = 3$

$x ** = 2$

$\text{print}(x) \# 9$

Relational / Comparison Operators:-

- Used to compare values. They return a boolean value true or false.
- $==, >, <, !=, >=, <=$

Logical Operator:-

Logical operator returns a boolean value by evaluating boolean expression.

→ and

→ or

→ not.

Logical And:-

		o/p
⇒ true	true	true
⇒ true	false	false
⇒ false	true	false
⇒ false	false	false

Logical OR:-

		o/p
⇒ true	true	true
⇒ true	false	true
⇒ false	true	true
⇒ false	false	false

Logical Not:-

	o/p
true	False
false	true

Bitwise Operator:-

Bitwise operators are used to compare (binary) numbers.

⇒ AND (2)

The operator compares each bit and set it to 1 if both are 1 otherwise it set to 0.

		O/P
0	0	0
0	1	0
1	0	0
1	1	1

Ex:- 6 & 3

$$6 = 110$$

$$3 = 011$$

$$\underline{010} \text{ } \textcircled{2}$$

⇒ OR (1)

The OR(1) operator compares each bit and set it to 1 if one or both is 1. Otherwise it set to 0.

		O/P
0	0	0
0	1	1
1	0	1
1	1	1

⇒ XOR (1)

The ^ operator compares each bit and set it to 1 if only one '1' is there. Otherwise it set to zero (if both are 0 or 1).

		O/P
0	0	0
0	1	1
1	0	1
1	1	0

NOT ($\sim$)

The $\sim$ operator inverts each bit (0 becomes 1 and 1 becomes 0)

	<u>o/p</u>
0	1
1	0

Ex: 3 becomes -4
~~3 = 0011~~

Left shift ($\ll$)

The $\ll$ operator inserts the specified number of 0's from the right and let the same amount of leftmost bits fall off.

Ex: $3 \ll 2$

3 =	0000000000000000000011
12 =	01100

Right shift ($\gg$) :-

The $\gg$ operator moves each bit the specified number of times to the right. Empty holes at the left are filled with 0's.

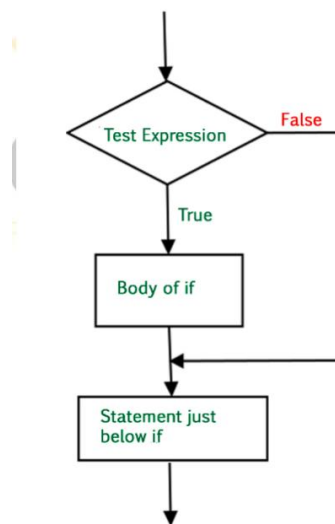
Ex: $8 =$ 0000 0000 0000 1000
 $2 =$ 0010

Conditional statements

If Statement:

The if statement is the most simple decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not.

Flowchart of If Statement Let's look at the flow of code in the Python If statements



Syntax of If Statement:

```
if condition:  
    # Statements to execute if  
    # condition is true
```

Example:

```
# if statement example  
if 10 > 5:  
    print("10 greater than 5")  
print("Program ended")
```

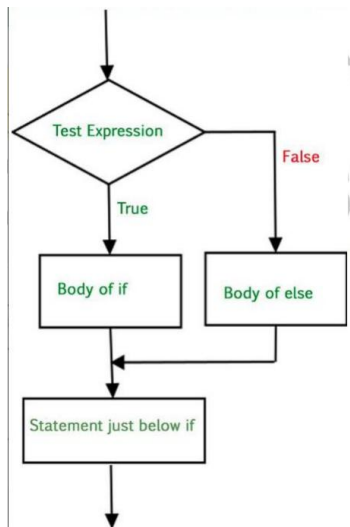
Output:

```
10 greater than 5  
Program ended
```

Python If Else Statement

The if statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't. But if we want to do something else if the condition is false, we can use the else statement with the if statement Python to execute a block of code when the Python if condition is false

Flowchart of If Else Statement Let's look at the flow of code in an if else Python statement.



Syntax of Python If-Else:

```

if (condition):
    # Executes this block if
    # condition is true
else:
    # Executes this block if
    # condition is false
  
```

Example:

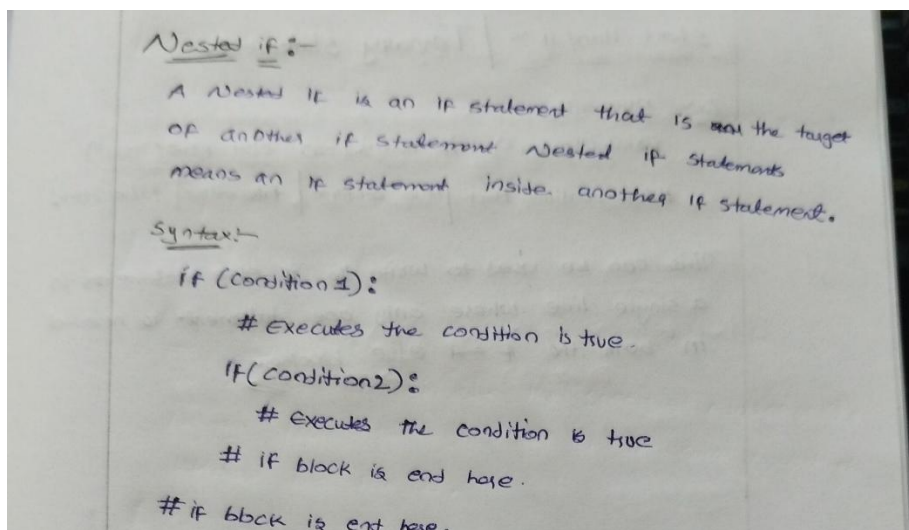
```

# if..else statement example
x = 3
if x == 4:
    print("Yes")
else:
    print("No")
  
```

Output: No

Python Nested If Statement

A nested if is an if statement that is the target of another if statement. Nested if statements mean an if statement inside another if statement.



Elif statement:-

Here, a user can decide among multiple options. The if statements are executed from the top down.

Syntax:-

```
if (condition):  
    statement
```

```
elif (condition):  
    statement
```

```
else:  
    statements
```

Note:- if statement cannot be empty use pass statement to avoid getting an error.

```
if (condition):  
    pass.
```

Short Hand if:- Ternary statement-

Ex:- num = 11
print("even") if num % 2 == 0 else print("odd")
true condition block | true condition | false condition | false block.

This can be used to write the if-else statements in a single line where only one statement is needed in both the if and else blocks.

LOOPS

For loop:-

A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set or a string).

- This is less like the for keyword in other programming language, and works more like an iterator method as found in other object oriented programming languages.

Ex:- # print each fruit in a fruit list
fruits = ["apple", "banana", "cherry"]
for x in fruits:
 print(x)

Ex:- # Looping through a string
for x in "banana":
 print(x)

⇒ Break:- With the break statement we can stop the loop before it has looped through all the items.

Ex:- fruits = ["apple", "banana", "cherry"]
for x in fruits:
 print(x)
 if x == "banana":
 break.

⇒ Continue:- It can stop the current iteration of the loop, and continue with the next.

Ex:- fruits = ["apple", "banana", "cherry"]
for x in fruits:
 if x == "banana":
 continue
 print(x)

The range() function:-

The range function returns a sequence of numbers, starting from 0 by default increments by 1 (by default), and ends at a specified number.

Examples:-

1. `for x in range(6):`
 `print(x)` $\downarrow$ end number
O/P:- 0, 1, 2, 3, 4, 5

Example-2:-

`for x in range(2, 6):`
 `print(x)` $\downarrow$ $\downarrow$
 start end.

Example-3:-

`for x in range(3, 30, 2):`
 `print(x)` $\downarrow$ $\downarrow$ $\downarrow$
 start end Increment.

Nested Loops:-

- A nested loop is a loop inside a loop.
- The inner loop will be executed one time for each iteration of the "outer loop".

Example:-

`adj = ["red", "big", "tasty"]`

`fruits = ["apple", "banana", "cherry"]`

`for x in adj:`

`for y in fruits:`

`print(x, y)`

While loop:-

While loop is used to execute a block of statements repeatedly until a given condition is satisfied. When the condition becomes false, the line immediately after the loop in the program is executed.

Syntax:-

```
while Expression:  
    Statement(s)
```

Example:-

```
count = 0  
while (count < 3):  
    count = count + 1  
    print("Hello")
```

Example:-

```
for i in range(1, 6, 1): # 1 2 3 4 5  
    for j in range(1, 11, 1): # 1 2 3 4 5 6 7 8 9 10  
        print(i, "x", j, "=", i*j)
```

Formatted String:-

```
name = "Abhi"  
print("hello {x}") # hello Abhi
```

↓
formatted string